

Contents

NEXIS — Feasibility, Scope and Dataset Strategy	1
0. Executive Summary	1
1. Verdict on Feasibility	2
2. The Three Genuinely Hard Problems	4
3. Dataset Strategy — the most important section	6
4. Research Framing — what to sharpen	9
5. Recommended Revised Scope	10
6. Risks That Are Not About Code	11
7. What I Would Change in the Blueprint	12
8. Final Assessment	13
Sources	13

NEXIS — Feasibility, Scope and Dataset Strategy

An honest engineering and research assessment of the Project Master Blueprint

0. Executive Summary

The NEXIS blueprint describes a coherent, genuinely capstone-worthy system. Nothing in it is technically impossible, and the intellectual core — early detection of *coordinated* fraud structures in a time-evolving heterogeneous graph — is a real research question with a real gap in the literature.

Three things, however, will determine whether this becomes a strong capstone or an unfinished one:

1. **Scope.** As written, the blueprint contains roughly three separate projects (a research ML project, a full-stack product, and an MLOps platform). The research project is the one that earns the grade and the paper. The other two are supporting cast and must be sized accordingly.
2. **Data.** The blueprint relies exclusively on a self-built synthetic generator. This creates a circularity that a reviewer will immediately attack: *you injected the patterns, so of course your model finds them*. This is the single biggest threat to the credibility of the results, and it is fixable cheaply.
3. **Novelty framing.** “GNN for fraud detection” is a saturated area. The blueprint acknowledges this (Section 9) but the six listed novelty claims are not equally defensible. One of them is genuinely publishable; the others are supporting evidence.

The rest of this document works through each of these.

1. Verdict on Feasibility

1.1 The short answer

Feasible as a research prototype. Not feasible as the full production platform described.

The blueprint’s own Section 18 roadmap lists 14 phases, of which four are marked “very hard” and seven “hard.” That is an accurate self-assessment. What the roadmap does not say is that several of those phases are *research* tasks with unbounded time, not engineering tasks with estimable time. Those are the ones that must be either de-risked or deferred.

1.2 Phase-by-phase risk assessment

The table below re-scores each blueprint phase. “Risk” here means *risk of consuming far more time than budgeted*, not difficulty in the abstract.

Phase	Deliverable	Blueprint difficulty	Real risk	Notes
0	Literature survey	medium	Low	Time-boxed. Do it, do not perfect it.
1	Data schema + EDA	medium	Low	Straightforward once the dataset is chosen.
2	Synthetic generator	hard	Medium	Buildable, but scope creep magnet. See §3.2.
3	Tabular baselines	medium	Low	A weekend of work. Highest value per hour in the project.
4	Graph construction	hard	Medium	The schema design is the hard part, not the code.
5	GCN / GraphSAGE / GAT	hard	Low-Medium	PyG makes this far easier than it sounds.
6	Temporal modeling	very hard	HIGH	The leakage-free implementation is where projects die. See §2.1.

Phase	Deliverable	Blueprint difficulty	Real risk	Notes
7	Fraud-ring detection	very hard	HIGHEST	This is set/subgraph prediction, not node classification. See §2.2.
8	XAI + evidence	hard	Medium	SHAP is easy; GNNExplainer on a temporal het-graph is not.
9	Drift + adaptation	very hard	HIGH	Requires a whole second experimental protocol. See §2.3.
10	AI investigator	hard	Low-Medium	Retrieval-first design makes this tractable. Do not over-engineer.
11	Backend + databases	hard	Low	Only if scoped to FastAPI + Postgres. Neo4j doubles it.
12	React dashboard	hard	Medium	Time sink. Budget it explicitly or it eats the research time.
13	Ablation + stress tests	very hard	Medium	Not conceptually hard — just needs a disciplined harness.
14	Paper + demo	hard	Medium	Underestimated by nearly everyone. Budget 3 full weeks.

1.3 The load-bearing observation

Phases 3, 4, 5 and 13 — baselines, graph construction, static GNNs, and the evaluation harness — are the *entire* scientific spine of the project. If those four are done rigorously and nothing else ships, you still have a defensible capstone and a submittable short paper.

Phases 6, 7 and 9 are what make it *novel*. They should be attempted, but each one should have a defined fallback (see §2).

Phases 10, 11, 12 are what make it *demonstrable*. They should be built last, deliberately thin, and timeboxed hard.

Everything else — Kafka, Redis, Neo4j, MLflow, the full Docker Compose stack — is optional infrastructure. The concept document’s own “Mistakes That Will Ruin the Results” list says exactly this: “*Adding Kafka/Redis/Neo4j before the core model works.*” Take your own advice.

2. The Three Genuinely Hard Problems

These deserve separate treatment because they are where the schedule will break.

2.1 Temporal modeling without leakage

Why it is hard. Leakage in temporal graph learning is insidious. It is not just “don’t shuffle your rows.” Every one of the following leaks future information into a past prediction, and every one of them is easy to do accidentally:

- Fitting a scaler, encoder, or imputer on the full dataset before splitting.
- Computing node degree, PageRank, or community membership on the *complete* graph and using it as a feature for an early-timestep prediction.
- Building the adjacency matrix once from all transactions, then slicing labels by time.
- Negative sampling from edges that do not exist *yet* but will.
- Early stopping on a validation set that sits chronologically after the test set.
- Aggregating a node’s neighbors without masking edges with timestamps later than the prediction time.

The fifth and sixth are the ones that catch strong students. A GNN message-passing step over a static adjacency matrix is *inherently* leaky on temporal data unless you explicitly mask.

How to de-risk it. Do not begin with a continuous-time model (TGN, TGAT, JODIE). Begin with **discrete snapshots**: partition time into windows, build a separate graph per window, train on windows 1..k, validate on k+1..m, test on m+1..T. This is far easier to verify as leakage-free — you can literally assert that no edge with timestamp > window_end exists in the graph object. Write that assertion as a unit test and run it in CI.

Only once snapshots work end-to-end should you attempt event-time message passing.

Fallback if it stalls. Snapshot-based temporal GNN with rolling-window features is a perfectly publishable configuration. Report it as the temporal model and discuss continuous-time as future work.

2.2 Fraud-ring detection

Why it is hard. Every other phase in the project is node-level or edge-level prediction — a well-posed supervised learning problem with a clean loss function. Ring detection is not. It is a *subgraph* prediction problem, and subgraph prediction has three structural difficulties:

1. **The label space is combinatorial.** There are exponentially many subgraphs. You cannot enumerate candidates and classify them.
2. **The ground truth is a set, not a class.** If the true ring is $\{A, B, C, D\}$ and you predict $\{A, B, C, E\}$, what is your score? You need a set-overlap metric (Jaccard, or precision/recall over ring membership) and you must define it before you start, not after you see results.
3. **Evaluation is genuinely ambiguous.** Is detecting 3 of 5 ring members a success? Is flagging a superset containing the ring a success? Reviewers will ask.

How to de-risk it. Use a two-stage design and be explicit that it is a heuristic-plus-learning hybrid, not an end-to-end learned detector:

- **Stage 1 (candidate generation):** cheap, high-recall, unsupervised. Connected components over high-risk edges, community detection (Louvain/Leiden) restricted to elevated-risk nodes, or motif enumeration (cycles up to length k , dense bipartite cores). This narrows exponential space to hundreds of candidates.
- **Stage 2 (candidate scoring):** supervised. Featurize each candidate subgraph (size, density, mean member risk, temporal concentration, amount-flow consistency, shared-device count, motif type) and train a plain classifier or a small GNN with graph-level readout to score it.

This is tractable, explainable, and honest. Define your metric as *ring-level precision/recall at a Jaccard overlap threshold of 0.5*, and report member-level precision/recall alongside.

Fallback if it stalls. Report network-risk *scoring* (every connected component gets a risk score, ranked) rather than ring *detection* (hard set membership). Ranking is easier to evaluate and still supports the early-warning story.

2.3 Drift adaptation

Why it is hard. Drift is not a model you build; it is an *experimental protocol* you design. To claim H5 (“drift-aware adaptation maintains performance better than a static model”) you need:

- A controlled way to *induce* drift with known ground truth (this is a real advantage of having your own generator).
- A detection mechanism (KS test, PSI, MMD, ADWIN) with a calibrated threshold — and calibrating it against false alarms is its own mini-project.

- An adaptation policy (retrain on window, retrain on all, weighted retrain, fine-tune) and a comparison across policies.
- A static baseline run over the identical drifted stream.
- Latency accounting: how long after the drift onset does the alarm fire, and how much performance is lost in that gap?

That is four experiments, not one.

How to de-risk it. Scope drift to a *single, clean, well-instrumented experiment*: one injected behavioral shift at a known time, one detector (PSI on the score distribution is the simplest defensible choice), one adaptation policy (retrain on the most recent window), compared against one static model. One figure: performance over time, static vs adaptive, with the drift onset and the detection point marked. That single figure is worth more than a half-finished continual-learning system.

Fallback if it stalls. Report drift *monitoring* (detection and alarm) without adaptation. Demonstrating that you can reliably detect degradation is a complete contribution on its own.

3. Dataset Strategy — the most important section

3.1 The problem with synthetic-only

The blueprint’s Section 12 proposes a configurable synthetic generator as the sole data source, justified by “real financial data are sensitive.” That justification is correct. The conclusion — use *only* synthetic — is not.

The failure mode is circular validation:

You define what a mule pattern looks like. You inject mule patterns matching that definition. You engineer features that capture that definition. Your model detects them with PR-AUC 0.97. You have proven that your generator and your detector agree with each other.

An examiner or reviewer will spot this in thirty seconds. It does not mean synthetic data is useless — it is *essential* for the controlled experiments (exact ground truth, tunable ring size, tunable prevalence, tunable camouflage, reproducible seeds). It just cannot be the only evidence.

3.2 The recommended three-tier data strategy

Tier 1 — Public real-world graph data (credibility anchor). One dataset with real labels, real structure and real temporal dynamics, used to show the method transfers beyond your own assumptions.

Tier 2 — Public large-scale synthetic AML data (scale + pattern realism). A dataset someone else generated, with laundering typologies someone else defined. This breaks the circularity — you did not design these patterns, so detecting them is evidence.

Tier 3 — Your own generator (controlled experiments). For everything that requires knobs: ring size sweeps, prevalence sweeps, camouflage intensity, drift injection, early-warning-time measurement. This is where your generator genuinely earns its place, because no public dataset lets you dial these.

Three tiers sounds like more work. It is actually *less* — Tiers 1 and 2 are downloads, and they let you keep your own generator simple instead of chasing realism it will never achieve.

3.3 Dataset options, assessed

IBM Transactions for Anti-Money Laundering (IT-AML / AMLworld) — *strongest recommendation for Tier 2* Generated by IBM Research with a multi-agent virtual world simulator; released publicly on Kaggle and documented in a peer-reviewed paper.

- **Scale:** six variants. Small (~2-5M transactions), Medium (~50-100M), Large (175-180M). Use a Small variant.
- **Laundering rate:** roughly 1 in 800 (HI variants) to 1 in 1,750 (LI variants) — realistically rare, which is exactly what you want.
- **Critical feature for NEXIS:** the generator injects **eight labeled laundering typologies** — fan-in, fan-out, scatter-gather, gather-scatter, simple cycle, bipartite, stack, and random. These map almost one-to-one onto the blueprint's Section 5 fraud-scenario table. This is enormously valuable: you get *labeled coordinated structures* that you did not design, which is precisely the evidence your ring-detection claim needs.
- **Also includes:** multi-bank, multi-currency, multiple transaction types, entity ownership graphs, and complete ground truth.
- **License:** data under CDLA-Sharing-1.0 (code Apache-2.0). Permissive for academic use; check the share-alike terms before redistributing derived data.
- **Verdict:** Use this. The typology labels alone justify it.

Elliptic++ — *strongest recommendation for Tier 1* Real Bitcoin blockchain data with human-annotated illicit/licit labels, extending the well-known Elliptic dataset.

- **Transactions component:** 203,769 nodes, 234,355 edges, 183 features, **49 time steps**. Labels: 4,545 illicit, 42,019 licit, 157,205 unknown.
- **Actors component:** 822,942 wallet addresses, 1,268,260 temporal interaction nodes, 2,868,964 address-to-address edges plus 1,314,241 address-transaction-address edges, 56 features, 49 time steps. Labels: 14,266 illicit, 251,088 licit, 557,588 unknown.
- **Why it fits NEXIS specifically:** the two components together give you a genuinely **heterogeneous** graph (transactions *and* actors, with three distinct edge types) that is also genuinely **temporal** (49 ordered steps). That is exactly the setting your research question is about. Very few public datasets offer both.
- **Caution:** the ~77% unknown-label rate is a real modeling constraint. Handle it explicitly — semi-supervised, or evaluate only on labeled nodes and say so. Do not silently treat unknown as licit.

- **Note:** original Elliptic is available directly in PyTorch Geometric, so a baseline is quick. Elliptic++ is hosted separately.
- **Verdict:** Use this as the reality check. It is also the most-cited benchmark in this literature, so results on it are directly comparable to published work — which strengthens your related-work section considerably.

PaySim / Sparkov / Credit Card Fraud (ULB) — optional, tabular only Useful only for Phase 3 tabular baselines and for sanity-checking your imbalance handling. They have little or no meaningful graph structure (PaySim’s is thin; ULB’s is anonymized PCA features with no entity identifiers at all). Do not build the graph story on these.

- **Verdict:** Optional. Use one if you want an extra baseline row in your results table. Skip if time is tight.

Your own generator — keep it, but shrink it The blueprint’s Section 12 function list (`generate_accounts`, `inject_mule`, `inject_cycle`, ...) is the right structure. But given Tiers 1 and 2, your generator no longer needs to be *realistic* — it needs to be *controllable*. That is a much easier target.

Prioritize the parameters that enable experiments you cannot run otherwise:

- ring size (for the “how large must a ring be to be detectable” curve)
- fraud prevalence (for the imbalance sensitivity curve)
- camouflage intensity (for the robustness story)
- delay distribution (for the early-warning-time measurement)
- drift onset time and magnitude (for H5)

Deprioritize: realistic merchant category distributions, realistic amount distributions, realistic geography. These add no experimental power and consume days.

3.4 How the three tiers map to your hypotheses

Hypothesis	Primary evidence	Supporting evidence
H1: graph > transaction-only on coordinated patterns	IBM AML (typology-labeled)	Own generator (ring-size sweep)
H2: temporal improves early detection	Own generator (exact ring-completion times)	Elliptic++ (49 time steps)
H3: heterogeneous relations reduce FPs	Elliptic++ (transactions + actors)	IBM AML (multi-bank, entity ownership)
H4: graph evidence improves interpretability	See §4.2 — reframe this	—
H5: drift adaptation beats static	Own generator (controlled drift)	—

Note how the own-generator column is now doing *specific experimental work* rather than carrying the whole project. That is the correct role for it.

4. Research Framing — what to sharpen

4.1 Novelty: pick one claim and defend it hard

Section 9 of the blueprint lists six novelty claims. Assessed honestly:

Claim	Assessment
Early detection of coordinated rings rather than transaction classification	Strong. This is the real contribution.
Temporal + heterogeneous instead of frozen account graph	Moderate — done before, but not usually together with ring-level evaluation.
Drift-aware adaptation	Moderate — well-studied generally, less so in graph fraud.
Evidence-centered explanations	Weak as novelty; strong as <i>engineering</i> . Present as a system feature.
Controllable synthetic fraud generator	Weak as novelty (AMLSim, IT-AML exist). Present as methodology.
Early-warning time at fixed FP budget	Strong. This is the evaluation contribution.

The paper’s contribution should be stated as one sentence combining rows 1 and 6:

We evaluate coordinated financial fraud detection by early-warning time at a fixed false-positive budget, and show that temporal heterogeneous graph learning detects fraud rings measurably earlier than transaction-level and static-graph baselines at equal analyst workload.

That is a defensible, specific, testable claim. Everything else in the blueprint becomes supporting apparatus for it.

4.2 H4 is not testable as written

“Graph evidence improves analyst interpretability” requires human subjects — a user study with actual analysts, an IRB-style protocol, and enough participants for a meaningful comparison. You almost certainly cannot run this.

Two honest alternatives:

- **Reframe as a proxy metric:** explanation *sparsity* (how few edges/nodes suffice to preserve the prediction), explanation *fidelity* (does the extracted subgraph reproduce the model’s score?), and explanation *stability* (do similar alerts get similar explanations?). These are standard, computable, and publishable.
- **Drop it and state why.** “We do not evaluate human interpretability, as this requires a user study beyond the scope of this work” is a perfectly respectable sentence in a limitations section.

Do not leave it as a hypothesis you cannot test.

4.3 One evaluation detail that will be challenged

The blueprint correctly insists on time-aware splits. Add one more discipline: **report variance**. Rare-event metrics on small positive counts are extremely noisy. With ~4,500 illicit transactions in Elliptic++ and a test split of maybe 20%, you are computing PR-AUC on a few hundred positives. Single-run numbers there are close to meaningless.

Run every configuration with at least 5 seeds and report mean $\pm$ standard deviation. If a claimed improvement is smaller than the seed-to-seed variance, it is not an improvement. This single practice separates capstones that survive questioning from ones that do not.

5. Recommended Revised Scope

5.1 What to cut outright

Component	Recommendation	Reason
Kafka	Cut	Adds zero research value. Simulate streaming by replaying a sorted CSV.
Redis	Cut	Premature optimization. Nothing here needs a cache.
Neo4j	Defer to stretch goal	Postgres + NetworkX + PyG covers every need. Neo4j is a demo nicety.
MLflow	Downgrade	A structured results/ directory with JSON configs and seeds achieves reproducibility. Adopt MLflow only if it costs under a day.
Full Docker Compose stack	Simplify	One Dockerfile that reproduces the training environment. That is what reproducibility requires.

Cutting these recovers an estimated three to four weeks.

5.2 A revised, realistic phase ordering

Block A — Foundation (weeks 1-5). Non-negotiable. Literature survey (timeboxed to 1 week) → dataset acquisition and EDA on IBM AML + Elliptic++ → **the evaluation harness first** (time-aware splitter, metric suite, seed management, results logging) → tabular baselines.

Building the evaluation harness *before* the models is the most important reordering in this document. Every subsequent model plugs into it, every result is automatically comparable, and leakage bugs surface immediately rather than in month five.

Block B — Graph core (weeks 6-10). Graph schema and construction with leakage assertions as unit tests → NetworkX structural analysis → homogeneous GNN (GCN/GraphSAGE/GAT) → heterogeneous GNN. First real result: does graph beat tabular, and by how much, with what variance?

Block C — The research contribution (weeks 11-17). Snapshot temporal model → early-warning-time measurement → candidate generation + subgraph scoring for rings → your own generator, built *now*, scoped to the five parameters in §3.3 → controlled experiments.

Note that the generator moves from Phase 2 to Block C. You do not need it until you need controlled experiments, and by then you will know exactly what it must produce. Building it first means building it blind.

Block D — Robustness and explanation (weeks 18-21). SHAP on tabular → graph explanation on the best graph model → evidence packet assembly → the single drift experiment from §2.3 → ablations across all models.

Block E — Product layer (weeks 22-25). Timeboxed hard. FastAPI over the trained model → Postgres schema → React dashboard with three views (alert queue, graph explorer, explanation panel) → retrieval-first investigator.

If Block E runs over, ship a Streamlit dashboard instead of React. It takes two days rather than three weeks and demonstrates exactly the same research. Nobody grades the CSS.

Block F — Writing (weeks 26-28). Paper, README, architecture doc, limitations, demo rehearsal.

5.3 The minimum defensible project

If everything goes wrong — and something always does — this is what must exist:

- Reproducible data pipeline on one public dataset with a time-aware split
- Three tabular baselines with tuned thresholds
- One homogeneous and one heterogeneous GNN
- One temporal configuration (snapshots are sufficient)
- Full metric suite reported with 5-seed variance
- One ablation table
- SHAP explanations on the tabular model
- A working API endpoint and any UI at all
- Honest limitations section

That is a complete, respectable capstone. Treat it as the floor and build upward.

6. Risks That Are Not About Code

Risk	Likelihood	Mitigation
Compute limits on Elliptic++ actors graph (2.8M edges)	High	Neighbor sampling (GraphSAGE-style), or use the transactions component only for the heavy experiments. Colab Pro or a single mid-range GPU suffices for the transactions graph.
Frontend consumes research time	High	Timebox to 2 weeks. Streamlit fallback ready from day one.
Ring detection produces no clean result	Medium	Fallback to network risk <i>ranking</i> (§2.2). Decide the fallback trigger date in advance.
Paper written last, rushed	High	Write the related-work and method sections <i>during</i> Blocks B and C, not after.
Scope re-expands mid-project	High	Keep §5.1 as a signed decision. Adding Neo4j in month four is how projects fail.
Results are unimpressive	Medium	Negative results are publishable <i>if</i> the protocol is rigorous. This is why the harness comes first.

7. What I Would Change in the Blueprint

Concrete, minimal edits:

1. **Section 6 → add a data-source subsection.** Name IBM AML and Elliptic++ explicitly as primary sources; reposition the synthetic generator as the controlled-experiment instrument.
2. **Section 8 → insert “Stage 0: evaluation harness”** before Stage A. Everything else depends on it.
3. **Section 9 → reduce to two novelty claims** (early ring detection; early-warning-time-at-fixed-FPR evaluation). Move the rest to “system contributions.”
4. **Section 10 → rewrite H4** as explanation sparsity/fidelity/stability, or delete it.
5. **Section 11 → add “reported as mean $\pm$ std over 5 seeds”** to the non-negotiable box alongside time-aware splits.
6. **Section 15 → mark Kafka, Redis, Neo4j and MLflow as “stretch”** rather than recommended stack.
7. **Section 18 → reorder** per §5.2: harness before models, generator after graph core.

8. **Section 22** → add a “**minimum viable**” **column** next to “Done when...”, per §5.3.
-

8. Final Assessment

The blueprint is well-conceived and unusually self-aware — the “Mistakes That Will Ruin the Results” list in the companion concept document shows the right instincts, and the insistence on time-aware splits and baseline-first modeling is exactly right.

The project will succeed or fail on three decisions, all of which should be made now rather than discovered in month four:

First, use public data as the credibility anchor and your own generator as the experimental instrument — not the other way round.

Second, build the evaluation harness before the models. Every hour spent here returns several later, and it is the difference between results you can defend and results you have to caveat.

Third, cut the infrastructure. Kafka, Redis, Neo4j and a production Docker Compose stack are not what makes this research; they are what prevents it from being finished.

Do those three things and the answer to the research question — *can adaptive temporal heterogeneous graph learning detect coordinated fraud earlier and with fewer false positives than transaction-level baselines?* — will be a real answer, whichever way it comes out.

Sources

- IBM AML-Data repository — <https://github.com/IBM/AML-Data>
- Altman et al., *Realistic Synthetic Financial Transactions for Anti-Money Laundering Models* (arXiv:2306.16424) — <https://arxiv.org/html/2306.16424v1>
- IBM Transactions for Anti Money Laundering (Kaggle) — <https://www.kaggle.com/datasets/ea1b/transactions-for-anti-money-laundering-aml>
- Elliptic++ Dataset — <https://github.com/git-disl/EllipticPlusPlus>
- Elmougy & Liu, *Demystifying Fraudulent Transactions and Illicit Nodes in the Bitcoin Network for Financial Forensics* (KDD '23) — <https://dl.acm.org/doi/10.1145/3580305.359>
- Elliptic Bitcoin dataset in PyTorch Geometric — <https://kumo.ai/pyg/datasets/elliptic-bitcoin/>